

Day 35

1. Communication Skill Evaluation

Project Structure

interview_ai/

|

└─ communication_engine.py

└─ normalization.py

|

tests/

|

└─ test_communication.py

└─ test_communication_run.py

1. interview_ai/communication_engine.py

```
import re
```

```
# -----
```

```
# Filler Words
```

```
# -----
```

```
FILLER_WORDS = [
```

```
    "um",
```

```
    "uh",
```

```
    "like",
```

```
    "you know",
```

```
    "actually",
```

```
    "basically"
```

```
]
```

```
# -----
```

```
# Fluency Score
```

```
# -----
```

```
def score_fluency(text):
```

```
    sentences = re.split(r"[.!?]", text)
```

```
    valid = [
```

```
        s for s in sentences
```

```
        if len(s.split()) > 3
```

```
    ]
```

```
    if len(valid) >= 2:
```

```
        return 1.0
```

```
    elif len(valid) == 1:
```

```
        return 0.6
```

```
    return 0.3
```

```
# -----
```

```
# Grammar Score
```

```
# -----
```

```
def score_grammar(text):
```

```
    if (
        text
        and text[0].isupper()
        and text.strip().endswith(
            ".", "!", "?"
        )
    ):
        return 1.0
```

```
    elif len(text.split()) > 5:
        return 0.7
```

```
    return 0.4
```

```
# -----
```

```
# Vocabulary Score
```

```
# -----
```

```
def score_vocabulary(text):
```

```
    words = text.lower().split()
```

```
    unique = set(words)
```

```
ratio = len(unique) / (len(words) + 1)
```

```
if ratio > 0.6:
```

```
    return 1.0
```

```
elif len(unique) > 5:
```

```
    return 0.7
```

```
return 0.4
```

```
# -----
```

```
# Clarity Score
```

```
# -----
```

```
def score_clarity(text):
```

```
    length = len(text.split())
```

```
    if length > 12:
```

```
        return 1.0
```

```
    elif length > 6:
```

```
        return 0.7
```

```
    return 0.4
```

```
# -----
```

```
# Structure Score
```

```
# -----
```

```
def score_structure(text):
```

```
    text = text.lower()
```

```
    if (
```

```
        "because" in text
```

```
        or "for example" in text
```

```
    ):
```

```
        return 1.0
```

```
    elif len(text.split()) > 6:
```

```
        return 0.7
```

```
    return 0.4
```

```
# -----
```

```
# Filler Penalty
```

```
# -----
```

```
def filler_penalty(text):
```

```
    text = text.lower()
```

```
count = sum(  
    text.count(word)  
    for word in FILLER_WORDS  
)
```

```
return min(count * 0.1, 0.5)
```

```
# -----
```

```
# Final Communication Score
```

```
# -----
```

```
def calculate_communication_score(text):
```

```
    fluency = score_fluency(text)
```

```
    grammar = score_grammar(text)
```

```
    vocabulary = score_vocabulary(text)
```

```
    clarity = score_clarity(text)
```

```
    structure = score_structure(text)
```

```
    penalty = filler_penalty(text)
```

```
    score = (
```

```
        fluency * 0.2
```

```
        + grammar * 0.2
```

```
        + vocabulary * 0.2
```

```
        + clarity * 0.2
```

```
        + structure * 0.2
```

```
)
```

```
score = max(score - penalty, 0)
```

```
return {  
    "communication_score": round(score * 100, 2),  
    "breakdown": {  
        "fluency": round(fluency, 2),  
        "grammar": round(grammar, 2),  
        "vocabulary": round(vocabulary, 2),  
        "clarity": round(clarity, 2),  
        "structure": round(structure, 2),  
        "penalty": round(penalty, 2)  
    }  
}
```

2. Score Normalization

```
interview_ai/normalization.py
```

```
def normalize_score(  
    score,  
    min_val=0,  
    max_val=100  
):  
  
    normalized = (  
        (score - min_val)  
        / (max_val - min_val)  
    )
```

```
return round(
    normalized * 100,
    2
)
```

3. Test Scripts

```
from interview_ai.communication_engine import (
    calculate_communication_score
)
```

```
def test_communication():
```

```
    result = calculate_communication_score(
        "I worked on several backend projects using Python."
    )
```

```
    assert result["communication_score"] > 0
```

tests/test_communication_run.py

```
from interview_ai.communication_engine import (
    calculate_communication_score
)
```

```
text = (
    "I have experience in Python because "
    "I worked on backend systems and REST APIs."
)
```

```
result = calculate_communication_score(text)
```

```
print("\nCommunication Score Output\n")
```

```
print(result)
```

Run the Test

```
python -m tests.test_communication
```

Run the Demo

```
python -m tests.test_communication_run
```

Expected Output

```
{  
  'communication_score': 86.0,  
  'breakdown': {  
    'fluency': 0.6,  
    'grammar': 1.0,  
    'vocabulary': 1.0,  
    'clarity': 1.0,  
    'structure': 1.0,  
    'penalty': 0.0  
  }  
}
```